1) Цель

   Организация межсетевого взаимодействия средствами WinAPI и POSIX.

2) Задание

   a) Организовать взаимодействие типа клиент-сервер средствами WinAPI и POSIX в соответствии с индивидуальным заданием.

   b) Вычислить номер варианта по списку в журнале и сохранить его в файл TASKID.txt в репозитории.

   c) Выбрать индивидуальное задание в соответствии с номером варианта. По номеру варианта также определить:

      (i) протокол, с использованием которого должен происходить обмен данными между сервером и клиентом;

      (ii) операционную систему, под которую необходимо разработать приложение сервера;

      (iii) операционную систему, под которую необходимо разработать приложение клиента.

   d) В разделе Issues репозитория создать не менее трех задач. Например: "Разработка приложения сервера "Разработка приложения клиента "Отладка клиент-серверного взаимодействия". Последовательно выполнить эти задачи, написав код и разместив его в репозитории. Решению каждой задачи должен соответствовать свой отдельный коммит, который должен быть привязан к конкретной задаче. Итого в репозитории должно быть не менее трех коммитов. При необходимости создать дополнительные задачи.

   e) Код приложения клиента необходимо разместить в файле client.cpp в корне репозитория, код приложения сервера - в файле server.cpp там же. При необходимости использовать дополнительные заголовочные файлы. Код, предназначенный для выполнения в ОС Linux, должен собираться командной g++ client.cpp или g++ server.cpp.

   f) В репозитории имеется тест, проверяющий корректность оформления кода в соответствии с Google C++ Style Guide. Данный тест запускает линтер, который проверяет соответствие кода стандарту языка и правилам оформления (отступы, разделители, комментарии и т.п.). В некоторых случаях линтер может дать совет, как улучшить код используя общепринятые практики.

   g) Автоматическое тестирование работоспособности кода отсутствует. Необходимо загрузить рабочий код в репозиторий, а затем защитить лабораторную работу у преподавателя, продемонстрировав корректную совместную работу клиентского и серверного приложений.

h) Допускается выполнение задания не в полном объеме. Правила начисления рейтинга для этого случая приведены в разделе Рейтинг.

i) Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report.tex.

j) Согласно варианту №7:

   (i) Сервер - ОС Windows;

   (ii) Клиент - ОС Linux;

   (iii) Протокол - TCP.

   Приложение-клиент запрашивает у пользователя ввод двух строк: слово и имя текстового файла, расположенного на сервере. Эти строки передаются на сервер, который подсчитывает количество повторений указанного слова в выбранном текстовом файле. Полученное число возвращается клиенту.

3) Схема взаимодействия между клиентом и сервером

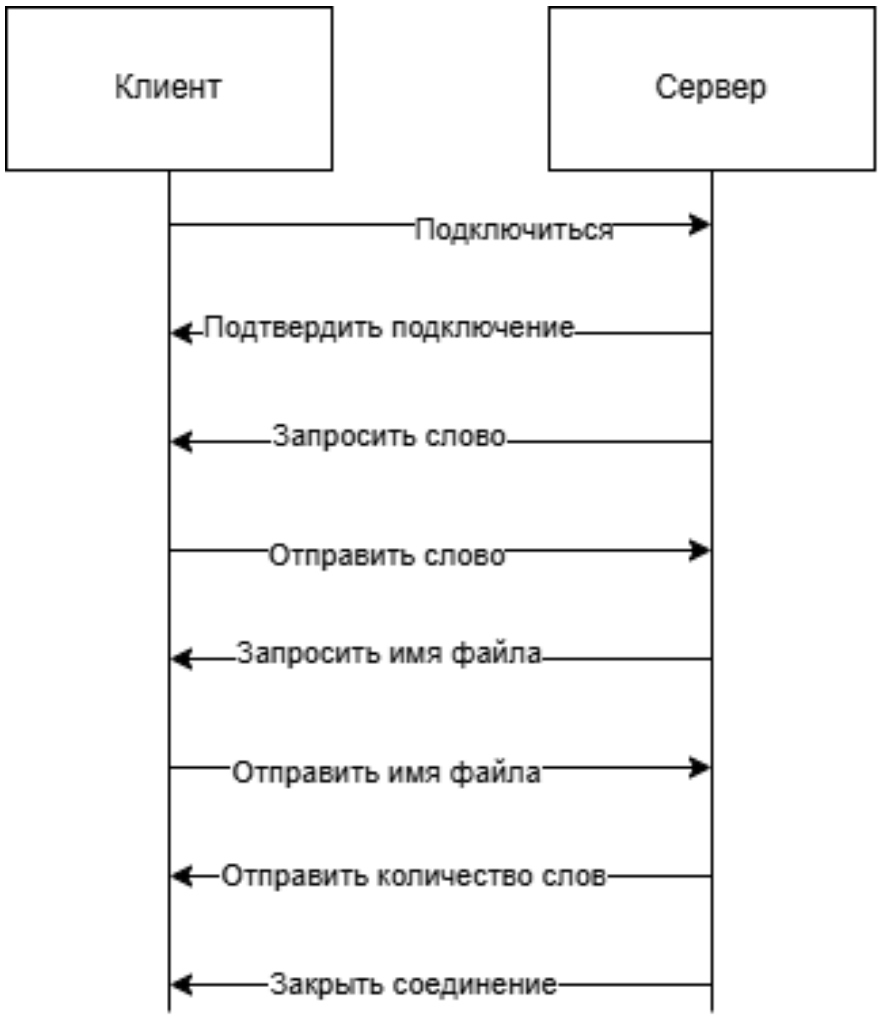


Рис. 1 – Взаимодейтсвие клиент-сервер

4) Результат выполнения работы

В процессе лабораторной работы был разработан сервер на ОС Windows, клиент на ОС Linux, их взаимодействие было запрограммировано через сокеты. Клиент-Сервер взаимодействие было проверенно и отлажено.

5) Исходный код

server.cpp

```
// Copyright 2025 Dabiil Voroshilov

#include <winsock2.h>
#include <windows.h>
#include <sstream>
#include <fstream>
#include <string>
#include <iostream>

using std::cout;
using std::endl;
using std::ifstream;
using std::string;
using std::istringstream;

bool bisServerRunning = true;
SOCKET ListenSocket = INVALID_SOCKET;

// Функиция обработки выхода
BOOL WINAPI ConsoleHandler(DWORD signal) {
    if (signal == CTRL_C_EVENT) {
        bisServerRunning = false;  // Ручная остановка сервера
        if (ListenSocket != INVALID_SOCKET) {
            closesocket(ListenSocket);
            ListenSocket = INVALID_SOCKET;
        }
    }
    return TRUE;
}

// Функция подсчёта количества в файле
int countWordsInFile(string FileName, string InWord) {
```

```
    ifstream File(FileName);
    if (!File.is_open()) {
        return -1;
    }

    int Count = 0;
    string CurrentWord;
    while (File >> CurrentWord) {
        if (CurrentWord == InWord) {
            Count++;
        }
    }

    return Count;
}

// Функция обработки клиента по сокету
DWORD WINAPI WorkForClientThread(LPVOID lpParam) {
    // Получаем сокет
    SOCKET InSocket = *reinterpret_cast<SOCKET*>(lpParam);
    delete lpParam;

    char RecBuffer[512];
    int bytes = recv(InSocket, RecBuffer, sizeof(RecBuffer)-1, 0);
    if (bytes > 0) {
        // Получаем мьютекс по имени
        HANDLE CoutMutex = CreateMutex(NULL, FALSE, "CoutMutex");

        string Word, FileName;
        RecBuffer[bytes] = '\0';
        istringstream iss(RecBuffer);
        iss >> Word >> FileName;

        WaitForSingleObject(CoutMutex, INFINITE);
        cout << "Клиент запросил найти в файле " << FileName <<
        " количество слов " << Word << endl;
        ReleaseMutex(CoutMutex);

        int Count = countWordsInFile(FileName, Word);
        string Reply = std::to_string(Count);
```

```
        WaitForSingleObject(CoutMutex, INFINITE);
        cout << "Отправляем клиенту количество: " << Reply << endl;
        ReleaseMutex(CoutMutex);

        send(InSocket, Reply.c_str(), Reply.size(), 0);
    }

    closesocket(InSocket);

    return 0;
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, ".utf8");
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    SetConsoleCtrlHandler(ConsoleHandler, TRUE);

    if (argc < 2) {
        cout << "Не указан порт сервера" << endl;
        return 1;
    }

    // Инициализируем WinSock
    WSADATA wsaData;
    int iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if ( iResult != NO_ERROR ) {
        cout << "Не удалось запустить WSAStartup()" << endl;
        return 1;
    }

    // Создаём сокет
    ListenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if ( ListenSocket == INVALID_SOCKET ) {
        cout << "Ошибка создания сокета: " << WSAGetLastError() << endl;
        WSACleanup();
        return 1;
    }
```

```
// Привязка сокета
struct sockaddr_in service;
service.sin_family = AF_INET;
service.sin_addr.s_addr = inet_addr("192.168.56.1");
service.sin_port = htons(std::stoi(argv[1]));

if (bind(ListenSocket, reinterpret_cast<SOCKADDR*>(&service),
    sizeof(service) ) == SOCKET_ERROR) {
    cout << "Ошибка привязки сокета" << endl;
    closesocket(ListenSocket);
    WSACleanup();
    return 1;
}

// Прослушиваем сокет для подключений
if ( listen(ListenSocket, 1) == SOCKET_ERROR ) {
    cout << "Ошибка прослушивания сокета" << endl;
    closesocket(ListenSocket);
    WSACleanup();
}

// Клиентский сокет
SOCKET ClientAcceptSocket = SOCKET_ERROR;



// Инициализируем мьютекс
HANDLE CoutMutex = CreateMutex(NULL, FALSE, "CoutMutex");
if (CoutMutex == NULL) {
    cout << "Не удалось создать мьютекс: " << GetLastError() << endl;
    closesocket(ListenSocket);
    WSACleanup();
    return 1;
}

// Принимает клиентов
cout << "Ожидаем клиентов для подсоединения..." << endl;
while (bisServerRunning) {
    ClientAcceptSocket = accept(ListenSocket, NULL, NULL);
    if (ClientAcceptSocket == INVALID_SOCKET) {
        break;
    }
```

```
        // Обрабатываем клиента
        SOCKET* Client = new SOCKET(ClientAcceptSocket);
        CreateThread(NULL, 0, WorkForClientThread, Client, 0, NULL);
        // Идём дальше для принятия новых клиентов
        ClientAcceptSocket = SOCKET_ERROR;
    }

    WSACleanup();
    CloseHandle(CoutMutex);

    return 0;
}
```

client.cpp

```
// Copyright 2025 Daniil Voroshilov
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <iostream>
#include <string>

using std::cout;
using std::endl;
using std::string;
using std::cin;

int main(int argc, char* argv[]) {
    if (argc < 3) {
        cout << "Не указан хост или порт" << endl;
        return 1;
    }


    // Инициализируем сокет
    int sock;
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
```

```
    cout << "Ошибка создания сокета" << endl;
    return 1;
}

// Соединяемся с сервером
sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(std::stoi(argv[2]));
if (inet_pton(AF_INET, argv[1], &addr.sin_addr) <= 0) {
    cout << "Ошибка преобразования хоста " << argv[1] << endl;
    close(sock);
    return 1;
}

if (connect(sock, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) < 0) {
    cout << "Ошибка соединения с " << argv[1] << ":" << argv[2] << endl;
    close(sock);
    return 1;
}

string Word;
string FileName;
cout << "Введите слово для поиска: " << endl;
cin >> Word;
cout << "Введите имя файла, в котором будет поиск: " << endl;
cin >> FileName;

string FullRequest = Word + ' ' + FileName;
send(sock, FullRequest.c_str(), FullRequest.size(), 0);

char ReqBuffer[512];
int recieve_bytes = recv(sock, &ReqBuffer, sizeof(ReqBuffer)-1, 0);

if (recieve_bytes > 0) {
    ReqBuffer[recieve_bytes] = '\0';
    string ServerOutput(ReqBuffer);

    cout << "Слово " << Word << " встречается в файле " <<
    FileName << " в количестве " << ServerOutput << endl;
}
```

```
    close(sock);

    return 0;
}
```

6) Выводы

В процессе выполнения лабораторной работы были получены практические навыки применения сокетов в разработке клиент-сервер взаимодействия между ОС Windows и Linux средствами POSIX и WinApi.